function test_bitwise_with_bool
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: UnaryOp(~) { Nop(Identifier) [var:a] }
B0 (exit)
Nop(exit)
function test_logical_with_int
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: BinaryOp(LogicExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: UnaryOp(!) { Nop(Identifier) [var:a] }
B0 (exit)
Nop(exit)
function test_array_wrong_index_type
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:val]
rhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
B0 (exit)
Nop(exit)
function test_array_wrong_element_type
B1 (entry)
Assign(=)
lhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
rhs: Nop(Identifier) [var:val]
B0 (exit)
Nop(exit)
function test_call_uint_with_int
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:res]
rhs: Call(helper_uint) { Nop(Identifier) [var:a] }
B0 (exit)
Nop(exit)
function test_call_mixed_types
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:res]
rhs: Call(helper_two_args) { Nop(Identifier) [var:b] | Nop(Identifier) [var:a] }
B0 (exit)
Nop(exit)
function test_call_missing_args
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:res]
rhs: Call(helper_int) { Nop(Identifier) [var:a] }
B0 (exit)
Nop(exit)
function test_call_wrong_arg_count
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:res]
rhs: Call(helper_int) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
B0 (exit)
Nop(exit)
function test_call_wrong_arg_type
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:res]
rhs: Call(helper_int) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function helper_two_args
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:r]
rhs: Nop(Identifier) [var:a]
B0 (exit)
Nop(exit)
function helper_uint
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:r]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:x] | Nop(Literal) [const:1] }
B0 (exit)
Nop(exit)
function helper_int
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:r]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:x] | Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function test_comparison_type_mismatch
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_byte_int_conflict
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: Nop(Identifier) [var:b]
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_uint_ulong_conflict
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: Nop(Identifier) [var:b]
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_int_long_conflict
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: Nop(Identifier) [var:b]
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_int_uint_conflict
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: Nop(Identifier) [var:b]
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)